Design a Movie Ticket Booking System

An online movie ticket booking system facilitates the purchasing of movie tickets to its customers. E-ticketing systems allow customers to browse through movies currently playing and book seats, anywhere and anytime.

movie
TICKET
Booking

Requirements and Goals of the System

Our ticket booking service should meet the following requirements:

1. It should be able to list the cities where affiliate cinemas are located.
2. Each cinema can have multiple halls and each hall can run one movie show at a time.
3. Each Movie will have multiple shows.
4. Customers should be able to search movies by their title, language, genre, release date, and city name.
5. Once the customer selects a movie, the service should display the cinemas running that movie and its available shows.
6. The customer should be able to select a show at a particular cinema and book their tickets.
7. The service should show the customer the seating arrangement of the cinema hall. The customer should be able to select multiple seats according to their preference.
8. The customer should be able to distinguish between available seats and booked ones.
9. The system should send notifications whenever there is a new movie, as well as when a booking is made or canceled.
10. Customers of our system should be able to pay with credit cards or cash.
11. The system should ensure that no two customers can reserve the same seat.
12. Customers should be able to add a discount coupon to their payment.

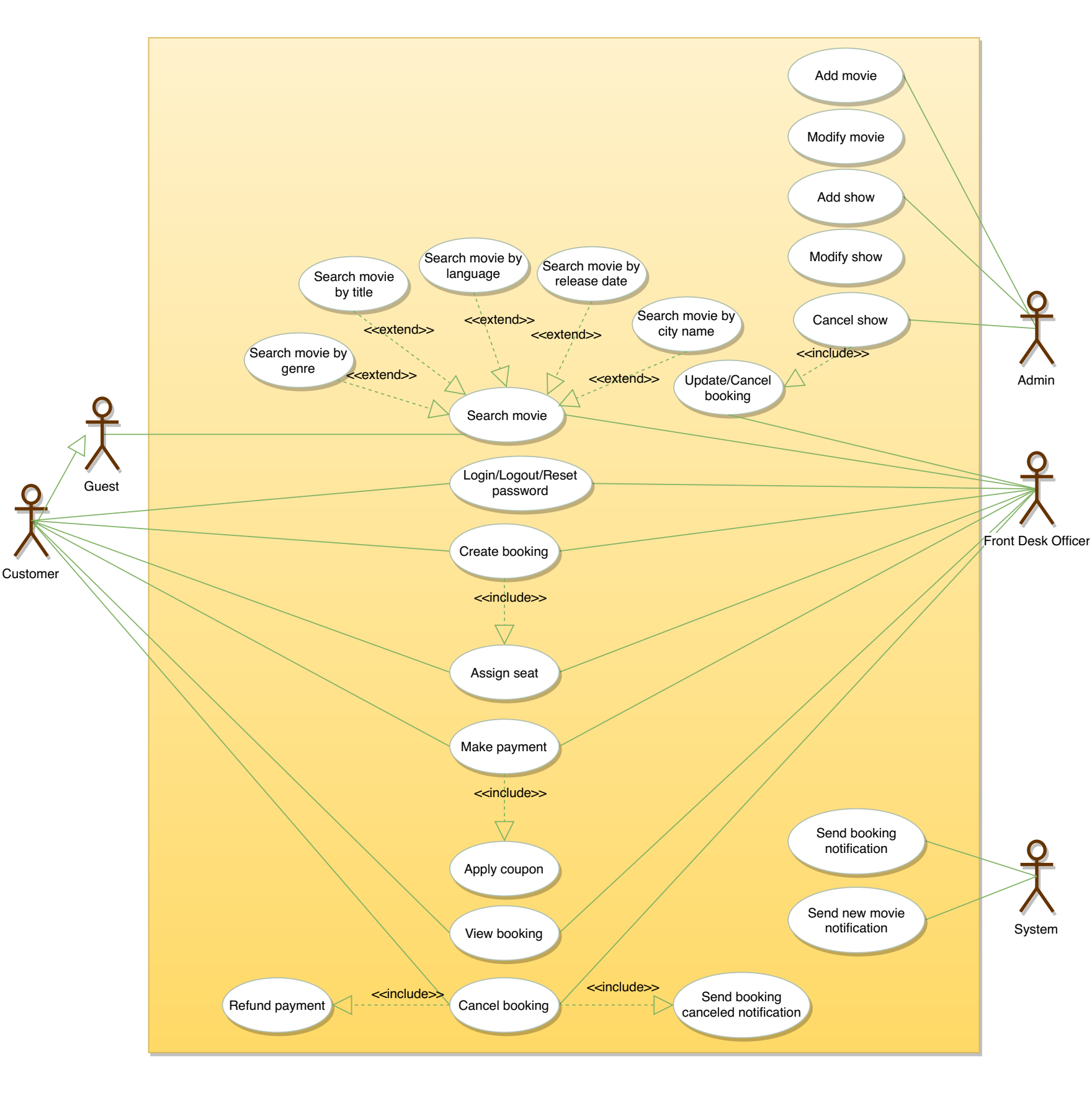
Use case diagram

We have five main Actors in our system:

- **Admin**: Responsible for adding new movies and their shows, canceling any movie or show, blocking/unblocking customers, etc.
- **FrontDeskOfficer**: Can book/cancel tickets.
- **Customer**: Can view movie schedules, book, and cancel tickets.
- **Guest**: All guests can search movies but to book seats they have to become a registered member.
- **System**: Mainly responsible for sending notifications for new movies, bookings, cancellations, etc.

Here are the top use cases of the Movie Ticket Booking System:

- **Search movies**: To search movies by title, genre, language, release date, and city name.
- **Create/Modify/View booking**: To book a movie show ticket, cancel it or view details about the show.
- **Make payment for booking**: To pay for the booking.
- **Add a coupon to the payment**: To add a discount coupon to the payment.
- **Assign Seat**: Customers will be shown a seat map to let them select seats for their booking.
- **Refund payment**: Upon cancellation, customers will be refunded the payment amount as long as the cancellation occurs within the allowed time frame.


Add movie
Modify movie
Add show
Modify show
Cancel show
Search movie by title
Search movie by language
Search movie by release date
Search movie by city name
Search movie by genre
<<extend>>
<<include>>
Update/Cancel booking
Search movie
Login/Logout/Reset password
Create booking
Assign seat
Make payment
Apply coupon
View booking
Cancel booking
Refund payment
Send booking canceled notification
Send booking notification
Send new movie notification
Guest
Customer
Admin
Front Desk Officer
System

Class diagram

Here are the main classes of the Movie Ticket Booking System:

- **Account**: Admin will be able to add/remove movies and shows, as well as block/unblock accounts. Customers can search for movies and make bookings for shows. FrontDeskOffice can book tickets for movie shows.

- **Guest**: Guests can search and view movies descriptions. To make a booking for a show they have to become a registered member.

- **Cinema**: The main part of the organization for which this software has been designed. It has attributes like 'name' to distinguish it from other cinemas.

- **CinemaHall**: Each cinema will have multiple halls containing multiple seats.

- **City**: Each city can have multiple cinemas.

- **Movie**: The main entity of the system. Movies have attributes like title, description, language, genre, release date, city name, etc.

- **Show**: Each movie can have many shows; each show will be played in a cinema hall.

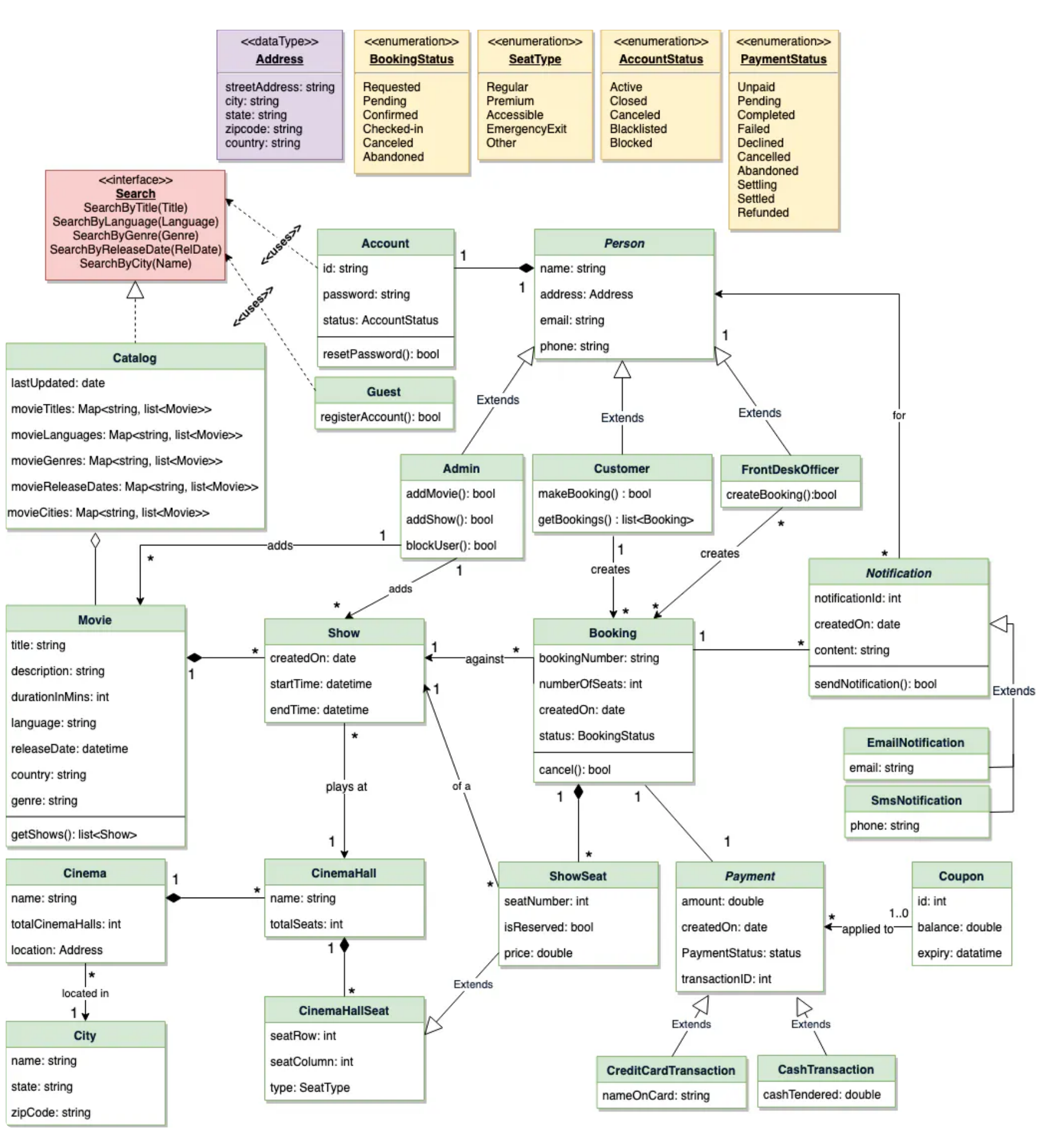
- **CinemaHallSeat**: Each cinema hall will have many seats.

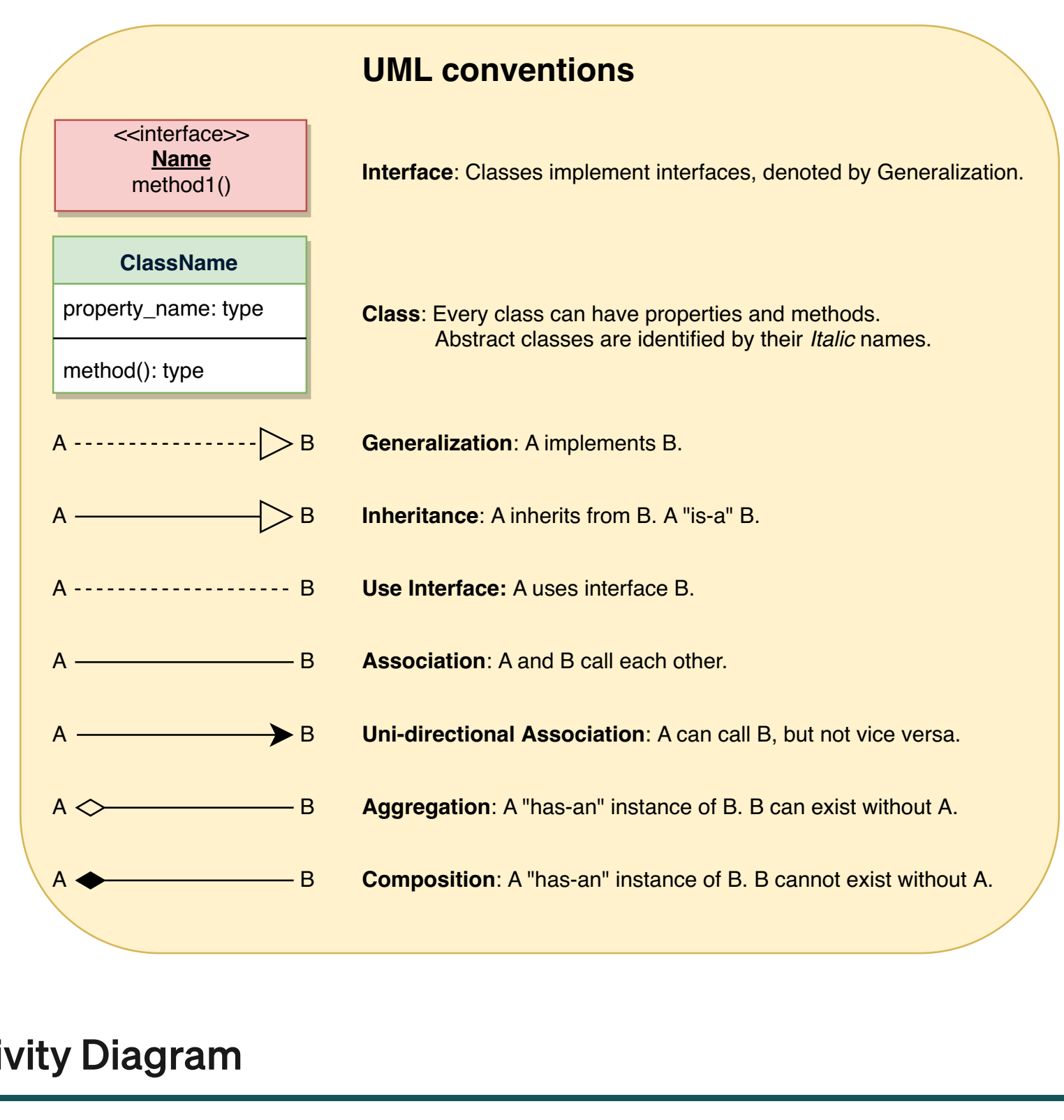
- **ShowSeat**: Each ShowSeat will correspond to a movie Show and a CinemaHallSeat. Customers will make a booking against a ShowSeat.

- **Booking**: A booking is against a movie show and has attributes like a unique booking number, number of seats, and status.

- **Payment**: Responsible for collecting payments from customers.

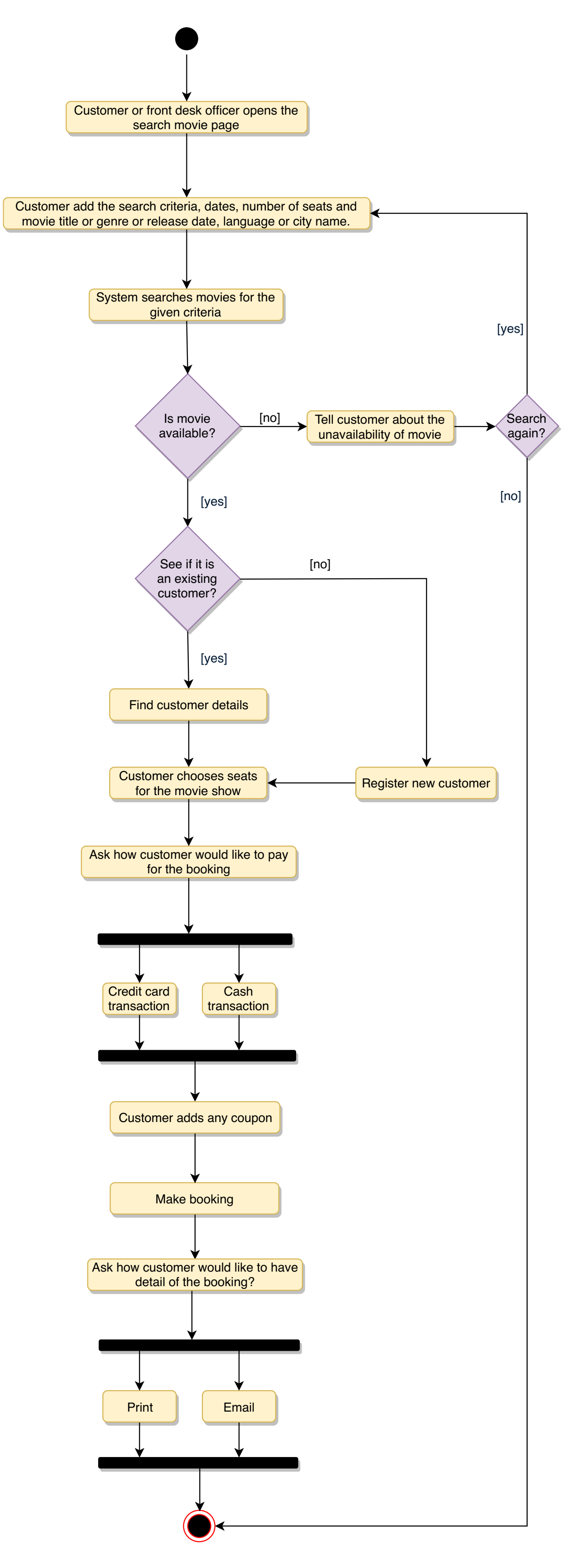
- **Notification**: Will take care of sending notifications to customers.


<<dataType>> Address
streetAddress: string
city: string
state: string
zipcode: string
country: string
<<enumeration>> BookingStatus
Requested
Pending
Confirmed
Checked-in
Canceled
Abandoned
<<enumeration>> SeatType
Regular
Premium
Accessible
EmergencyExit
Other
<<enumeration>> AccountStatus
Active
Closed
Canceled
Blacklisted
Blocked
<<enumeration>> PaymentStatus
Unpaid
Pending
Completed
Failed
Declined
Cancelled
Abandoned
Settling
Settled
Refunded
<<interface>> Search
SearchByTitle(Title)
SearchByLanguage(Language)
SearchByGenre(Genre)
SearchByReleaseDate(RelDate)
SearchByCity(Name)
<<uses>>
Account
id: string
password: string
status: AccountStatus
resetPassword(): bool
Person
name: string
address: Address
email: string
phone: string
Guest
registerAccount(): bool
Catalog
lastUpdated: date
movieTitles: Map<string, list<Movie>>
movieLanguages: Map<string, list<Movie>>
movieGenres: Map<string, list<Movie>>
movieReleaseDates: Map<string, list<Movie>>
movieCities: Map<string, list<Movie>>
Extends
Admin
addMovie(): bool
addShow(): bool
blockUser(): bool
Customer
makeBooking() : bool
getBookings() : list<Booking>
FrontDeskOfficer
createBooking():bool
for
Notification
notificationId: int
createdOn: date
content: string
sendNotification(): bool
adds
creates
Movie
title: string
description: string
durationInMins: int
language: string
releaseDate: datetime
country: string
genre: string
getShows(): list<Show>
Show
createdOn: date
startTime: datetime
endTime: datetime
against
Booking
bookingNumber: string
numberOfSeats: int
createdOn: date
status: BookingStatus
cancel(): bool
EmailNotification
email: string
SmsNotification
phone: string
plays at
of a
Cinema
name: string
totalCinemaHalls: int
location: Address
CinemaHall
name: string
totalSeats: int
ShowSeat
seatNumber: int
isReserved: bool
price: double
Payment
amount: double
createdOn: date
PaymentStatus: status
transactionID: int
Coupon
id: int
balance: double
expiry: datatime
applied to
located in
City
name: string
state: string
zipCode: string
CinemaHallSeat
seatRow: int
seatColumn: int
type: SeatType
CreditCardTransaction
nameOnCard: string
CashTransaction
cashTendered: double
1
*
1..0

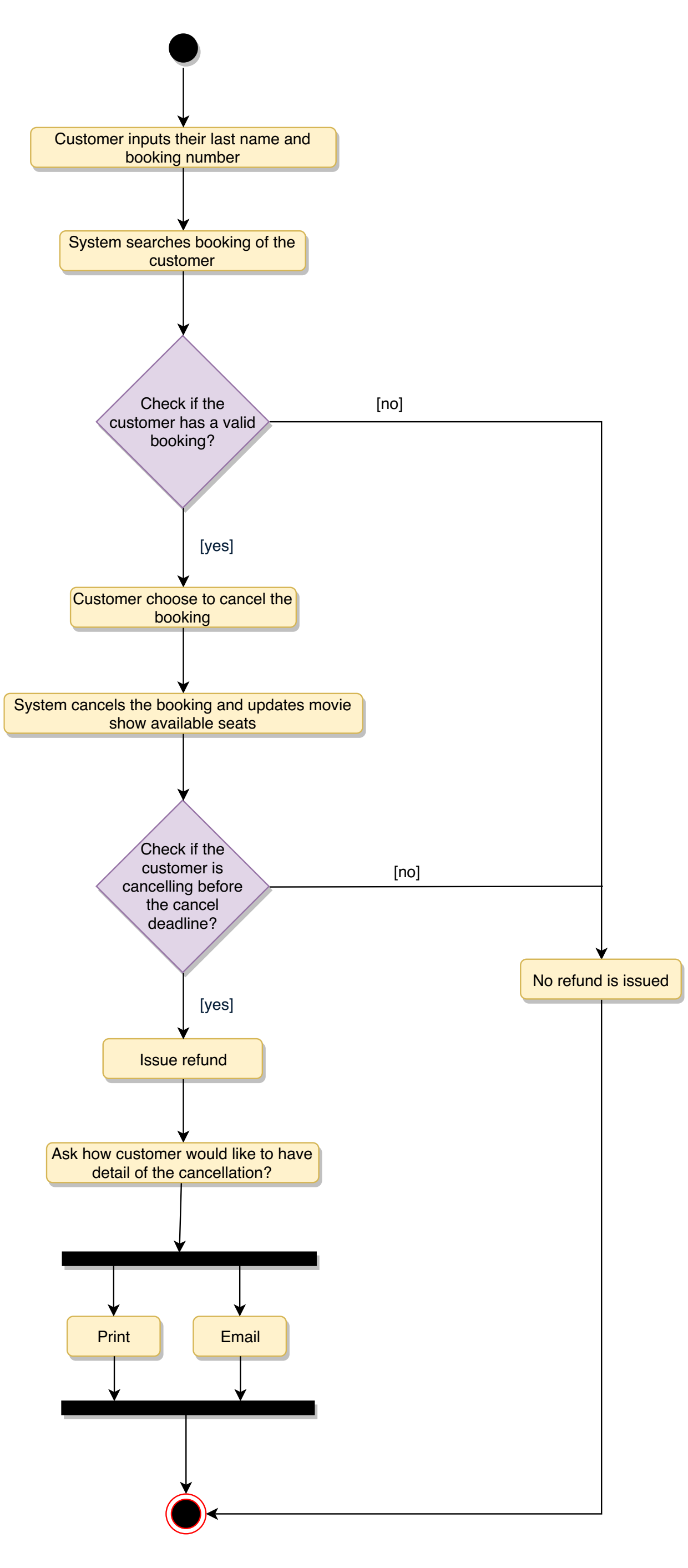

UML conventions
<<interface>>
Name
method1()
Interface: Classes implement interfaces, denoted by Generalization.
ClassName
property_name: type
method(): type
Class: Every class can have properties and methods. Abstract classes are identified by their Italic names.
A
B
Generalization: A implements B.
Inheritance: A inherits from B. A "is-a" B.
Use Interface: A uses interface B.
Association: A and B call each other.
Uni-directional Association: A can call B, but not vice versa.
Aggregation: A "has-an" instance of B. B can exist without A.
Composition: A "has-an" instance of B. B cannot exist without A.

Activity Diagram

- **Make a booking**: Any customer can perform this activity. Here are the steps to book a ticket for a show:


Customer or front desk officer opens the search movie page
Customer add the search criteria, dates, number of seats and movie title or genre or release date, language or city name.
System searches movies for the given criteria
Is movie available?
[no]
Tell customer about the unavailability of movie
Search again?
[yes]
[no]
[yes]
See if it is an existing customer?
[no]
[yes]
Find customer details
Customer chooses seats for the movie show
Register new customer
Ask how customer would like to pay for the booking
Credit card transaction
Cash transaction
Customer adds any coupon
Make booking
Ask how customer would like to have detail of the booking?
Print
Email

- **Cancel a booking**: Customer can cancel their bookings. Here are the steps to cancel a booking:


Customer inputs their last name and booking number
System searches booking of the customer
Check if the customer has a valid booking?
[no]
[yes]
Customer choose to cancel the booking
System cancels the booking and updates movie show available seats
Check if the customer is cancelling before the cancel deadline?
[no]
No refund is issued
[yes]
Issue refund
Ask how customer would like to have detail of the cancellation?
Print
Email

Code

Here are the high-level definitions for the classes described above.

Enums, data types, and constants: Here are the required enums, data types, and constants:

```java
public enum BookingStatus {
REQUESTED, PENDING, CONFIRMED, CHECKED_IN, CANCELED, ABANDONED
}

public enum SeatType {
REGULAR, PREMIUM, ACCESSIBLE, SHIPPED, EMERGENCY_EXIT, OTHER
}

public enum AccountStatus {
ACTIVE, BLOCKED, BANNED, COMPROMISED, ARCHIVED, UNKNOWN
}

public enum PaymentStatus {
UNPAID, PENDING, COMPLETED, FILLED, DECLINED, CANCELLED, ABANDONED, SETTLING, SETTLED, R
}

public class Address {
private String streetAddress;
private String city;
private String state;
private String zipCode;
```

Account, Customer, Admin, FrontDeskOfficer, and Guest: These classes represent the different people that interact with our system:

```java
// For simplicity, we are not defining getter and setter functions. The reader can
  // assume that all class attributes are private and accessed through their respective
  // public getter method and modified only through their public setter method.

  public class Account {
  private String id;
  private String password;
  private AccountStatus status;

  public boolean resetPassword();
  }

  public abstract class Person {
  private String name;
  private Address address;
  private String email;
  private String phone;

  private Account account;
  }

```

Show and Movie: A movie will have many shows:

```java
public class Show {
private int showId;
private Date createdOn;
private Date startTime;
private Date endTime;
private CinemaHall playedAt;
private Movie movie;
}

public class Movie {
private String title;
private String description;
private int durationInMins;
private String language;
private Date releaseDate;
private String country;
private String genre;
private Admin movieAddedBy;

private List<Show> shows;
public List<Show> getShows();
```

Booking, ShowSeat, and Payment: Customers will reserve seats with a booking and make a payment:

```java
public class Booking {
 private String bookingNumber;
 private int numberOfSeats;
 private Date createdOn;
 private BookingStatus status;

 private Show show;
 private List<ShowSeat> seats;
 private Payment payment;

 public boolean makePayment(Payment payment);
 public boolean cancel();
 public boolean assignSeats(List<ShowSeat> seats);
 }

 public class ShowSeat extends CinemaHallSeat{
 private int showSeatId;
 private boolean isReserved;
 private double price;
 }

```

City, Cinema, and CinemaHall: Each city can have many cinemas and each cinema can have many cinema halls:

```java
public class City {
private String name;
private String state;
private String zipCode;
}

public class Cinema {
private String name;
private int totalCinemaHalls;
private Address location;

private List<CinemaHall> halls;
}

public class CinemaHall {
private String name;
private int totalSeats;

private List<CinemaHallSeat> seats;
private List<Show> shows;
}
```

Search interface and Catalog: Catalog will implement Search to facilitate searching of products.

```java
public interface Search {
public List<Movie> searchByTitle(String title);
public List<Movie> searchByLanguage(String language);
public List<Movie> searchByGenre(String genre);
public List<Movie> searchByReleaseDate(Date relDate);
public List<Movie> searchByCity(String cityName);
}

public class Catalog implements Search {
HashMap<String, List<Movie>> movieTitles;
HashMap<String, List<Movie>> movieLanguages;
HashMap<String, List<Movie>> movieGenres;
HashMap<Date, List<Movie>> movieReleaseDates;
HashMap<String, List<Movie>> movieCities;

public List<Movie> searchByTitle(String title) {
return movieTitles.get(title);
}

public List<Movie> searchByLanguage(String language) {
return movieLanguages.get(language);
```

Concurrency

How to handle concurrency; such that no two users are able to book the same seat? We can use transactions in SQL databases to avoid any clashes. For example, if we are using SQL server we can utilize Transaction Isolation Levels to lock the rows before we update them. Note: within a transaction, if we read rows we get a write-lock on them so that they can't be updated by anyone else. Here is the sample code:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

BEGIN TRANSACTION;

    -- Suppose we intend to reserve three seats (IDs: 54, 55, 56) for ShowID=99
    Select * From ShowSeat where ShowID=99 && ShowSeatID in (54, 55, 56) && isReserved=0

    -- if the number of rows returned by the above statement is NOT three, we can return
failure to the user.
    update ShowSeat table ...
    update Booking table  ...

COMMIT TRANSACTION;
```

'Serializable' is the highest isolation level and guarantees safety from Dirty, Nonrepeatable, and Phantoms reads.

Once the above database transaction is successful, we can safely assume that the reservation has been marked successfully and no two customers will be able to reserve the same seat.

Here is the sample Java code:

```java
import java.sql.DriverManager;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.sql.ResultSet;

public class Customer extends Person {

public boolean makeBooking(Booking booking) {
List<ShowSeat> seats = booking.getSeats();
Integer seatIds[] = new Integer[seats.size()];
int index = 0;
for(ShowSeat seat : seats) {
seatIds[index++] = seat.getShowSeatId();
}

Connection dbConnection = null;
try {
dbConnection = getDBConnection();
dbConnection.setAutoCommit(false);
// 'Serializable' is the highest isolation level and guarantees safety from
```

← Previous
Design Stack Overflow

Next →
Design An ATM

☐ Mark as Completed